# Clase 26 — IPv4: el datagrama y las direcciones

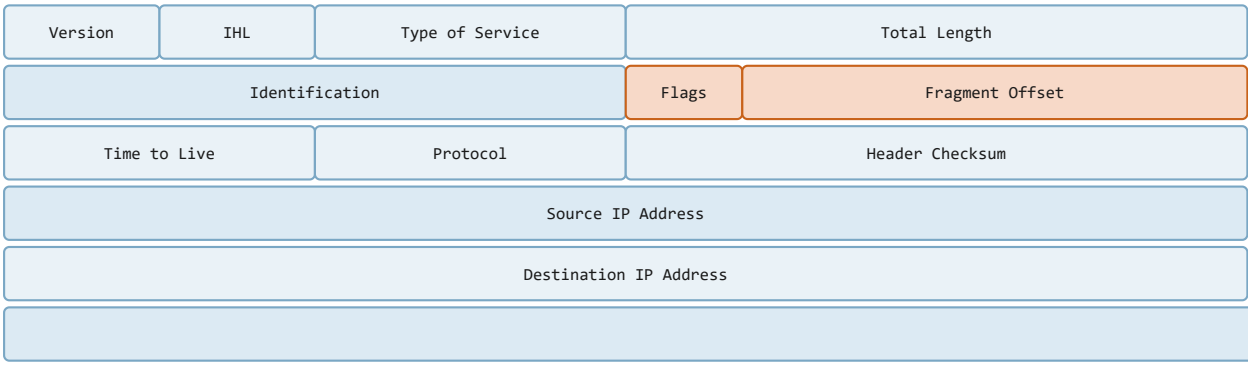
**Bloque:** D (Red) · **Semana:** 9 · **Duración estimada:** 110 min **Correlativa:** Clase 25 (conmutación en la red) · **Sigue con:** Clase 27 (subnetting, método paso a paso) **Material de referencia:** Tanenbaum cap. 5 (§5.4) · RFC 791, RFC 1122, RFC 1812, RFC 1878, RFC 4632, RFC 8200

---

## 🎯 Al terminar esta clase vas a poder

1. **Dibujar** el formato de la cabecera IPv4 campo por campo en filas de 32 bits, con el **tamaño en bits** de cada campo y un **ejemplo numérico** de cada uno.
2. **Explicar** para qué sirve el campo **IHL**, por qué la cabecera es de **tamaño variable**, y qué límite impone el campo *Longitud total*.
3. **Aplicar una máscara de subred bit a bit** sobre una dirección en binario, y obtener de ahí la **dirección de red**, el **rango de hosts** y la **dirección de broadcast**.
4. **Identificar la clase** (A/B/C/D/E) de una dirección, dar su **máscara por defecto** y decir **cuántos hosts** caben.
5. **Explicar** por qué las clases quedaron obsoletas, qué es **CIDR** y qué problema económico resolvió.
6. **Distinguir** los rangos **especiales** de IPv4 ( `0/8` , `127/8` , `169.254/16` , `224/4` , `255.255.255.255` ) y las **direcciones privadas** ( `10/8` , `172.16/12` , `192.168/16` ), y decir para qué sirve cada uno.
7. **Calcular** el **checksum** de una cabecera IPv4 con la suma del complemento a uno, y verificar que da `0xFFFF` .

### Cabecera IPv4, campo por campo




En naranja: los tres campos que hacen posible la fragmentación. El bit DF es el 2.º de Flags.

IHL = 5 significa cabecera de 20 bytes sin opciones.

### Como se aplica una mascara de subred, bit a bit

| IP | Mascara (/26) | AND = resultado |
|---|---|---|
| 11000000 | 11111111 | **11000000** |
| 10101000 | 11111111 | **10101000** |
| 00001010 | 11111111 | **00001010** |
| 00110111 | 11000000 | **00000000** |

**192.168.10.55 AND 255.255.255.192 = 192.168.10.64**

**Lo que resulta de ese AND:**

| | |
|---|---|
| direccion de red | 192.168.10.64 |
| primer host | 192.168.10.65 |
| ultimo host | 192.168.10.126 |
| broadcast | 192.168.10.127 |
| hostables | 62 |

**El prefijo /26 significa:**

26 bits de red + 6 bits de host

2^6 = 64 direcciones en la subred

64 - 2 = 62 hostables

El AND pone en 0 todos los bits de host: por eso el resultado es la direccion de RED, no una de host.

> **Prerequisito del examen:** `2010-03 Problema 1` (30 pts — tres subredes `/24` con R1 y R2: tablas de enrutamiento y por qué no se pueden renumerar agrupando), `2009-12 Problema 1` (tres clases C y su agregación), `2008-12 P2`, `2011-03 P2d`.

---

# PARTE 1 — TEORÍA

## 1.1 El datagrama IPv4 en su contexto

Antes de la cabecera hay que tener claro **dónde viaja**. El datagrama IP no viaja solo: viaja **encapsulado** dentro de una trama de capa 2.

```
┌──────────────────────────── Trama Ethernet (o de otro enlace) ──────┐
│  PREÁMBULO │  MAC destino 6 │ MAC origen 6 │ Ethertype 2 │  ...     │
│  ┌──────────────────────── Datagrama IP (20 a 60 bytes) ──────────┐   │
│  │  CABECERA IP: 20 bytes si IHL = 5                          │   │
│  │  DATOS: el segmento TCP (20 bytes de cabecera + datos)     │   │
│  └────────────────────────────────────────────────────────────┘   │
│  ... │  FCS 4 │                                                   │
└──────┴────────┴─────────────────────────────────────────────────────┘

Ethertype = 0x0800  →  "el contenido de esta trama es un datagrama IP"
Ethertype = 0x0806  →  "es un mensaje ARP"  (ver Clase 30)

MTU del enlace = la mayor trama que transmite sin fragmentar.
Ethernet: 1500 bytes.  Si la trama completa es 1518 bytes con la
cabecera de Ethernet, el datagrama IP disponible es 1500 bytes.
```

El **datagrama IP tiene un tamaño máximo de 65 535 bytes** (por el campo *Longitud total*, 16 bits), pero el **tamaño mínimo obligatorio es 576 bytes** (de `RFC 791` y `RFC 1122` §3.2.1), para que las redes con MTU chico no fragmenten. En la práctica el MTU manda: 1500 en Ethernet, 9000 (*jumbo frames*) en algunos datacenters — ojo, **9000 es una convención, no un estándar IEEE** — y 1280 como mínimo absoluto en IPv6.

## 1.2 La cabecera IPv4, campo por campo

La cabecera mínima son **20 bytes** (160 bits), y se dibuja siempre en **cinco filas de 32 bits**:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Version|  IHL  |    DSCP   |ECN|         Length                |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                         Id                                    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|      Flags     |    Fragment Offset    |     TTL     |Protocol  |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                       Header Checksum                         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                       Source Address                          |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Destination Address                        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

| # | Campo | Bits | Qué significa | Ejemplo numérico |
|---|---|---|---|---|
| 1 | **Versión** | 4 | Qué versión del protocolo IP es este datagrama. `0100` = 4. | `4` (IPv4). En IPv6 son los 4 primeros bits de una cabecera de 40 bytes. |
| 2 | **IHL** (*Internet Header Length*) | 4 | **Longitud de la cabecera en palabras de 32 bits**, no en bytes. Multiplicá por 4 para obtener los bytes. | `0101` = 5 → `5 × 4` = **20 bytes**. El mínimo es 5 (20 B); el máximo es 15 (60 B, con 40 bytes de opciones). |
| 3 | **DSCP** | 6 | *Differentiated Services Code Point*: **con qué prioridad** tratar el paquete (DiffServ). | `000000` = mejor esfuerzo (*best effort*), que es el valor por defecto. `101110` = EF (*Expedited Forwarding*), voz. |
| 4 | **ECN** | 2 | *Explicit Congestion Notification*: los 2 bits de la Clase 21. | `00` = No-ECT (el origen no usa ECN). `11` = CE (*Congestion Experienced*). |
| 5 | **Longitud total** (*Total Length*) | 16 | **Longitud de TODO el datagrama IP: cabecera + datos, en bytes.** Máx. 65 535. Mín. 20 (cabecera sola). | `0054` = 84 bytes: 20 de cabecera IP + 20 de cabecera TCP + 44 de datos. |
| 6 | **Identificación** (*Identification*) | 16 | Identifica **un grupo de fragmentos** del mismo datagrama original. Sólo importa si hay fragmentación. | `1C46` . Lo usan la fuente y los routers intermedios para reensamblar. |
| 7 | **Flags** | 3 | Tres banderas: bit de reserva (**siempre 0**), **DF** (*Don't Fragment*) y **MF** (*More Fragments*). | `100` = DF=1, MF=0 → "**no me fragmentes**" (comportamiento por defecto hoy). `001` = DF=0, MF=1 → "hay más fragmentos". |
| 8 | **Offset de fragmento** (*Fragment Offset*) | 13 | Posición **en bytes de los datos** de este fragmento dentro del datagrama original. Se cuenta en **unidades de 8 bytes**, así que los últimos 3 bits no se usan. | `0000` = este fragmento empieza en el byte 0 del payload original (no es un fragmento). `0001` = arranca en el byte 8. |
| 9 | **TTL** (*Time To Live*) | 8 | **Contador de saltos.** Cada router lo **decrementa en 1** antes de reenviar; al llegar a 0, descarta el paquete y manda un ICMP *Time Exceeded*. | `40` = 64 saltos. Windows empieza en 128, Linux en 64. |
| 10 | **Protocolo** | 8 | Qué protocolo va **encapsulado** dentro del campo de datos. | `06` = TCP · `11` = UDP · `01` = ICMP · `2E` = IPv6 dentro (túnel) · `08` = un protocolo de GRE. |
| 11 | **Checksum de cabecera** | 16 | **Suma del complemento a uno** de todos los bytes de la | `5958` (el cálculo completo está en §2.3). |

| # | Campo | Bits | Qué significa | Ejemplo numérico |
|---|---|---|---|---|
| | | | cabecera. **Sólo de la cabecera**: si cambia un dato, el TCP/UDP recalcula el suyo. | |
| 12 | **Dirección origen** | 32 | De quién es el datagrama. **Cambia en cada salto si hay NAT.** | `192.168.10.55` = `0xC0A80A37` = `11000000.10101000.00001010.00110111` |
| 13 | **Dirección destino** | 32 | A quién va el datagrama. **No cambia** al atravesar la red (salvo NAT). | `219.41.31.21` = `0xDB291F15` = `11011011.00101001.00011111.00010101` |

**El mismo datagrama, escrito en hexadecimal** (que es como lo vas a ver en un `wireshark`):

```
45 00 0054 1C46 4000 40 06 5958 C0A8 0A37 DB29 1F15
^^ ^^ ____ ____ ____ ^^ ^^ ____ ____ ____ ____
|  |  |    |    |  |  |  |    |    |    |
|  |  |    |    |  |  |  |    |    |    └─ destino 219.41.31.21
|  |  |    |    |  |  |  |    |    └────── origen   192.168.10.55
|  |  |    |    |  |  |  |    └─────────── checksum de cabecera = 0x5958
|  |  |    |    |  |  |  └──────────────── protocolo = 06 = TCP
|  |  |    |    |  |  └─────────────────── TTL = 0x40 = 64
|  |  |    |    |  └────────────────────── Flags/Offset = 0x4000 → DF=1
|  |  |    |    └───────────────────────── Identificación = 0x1C46
|  |  |    └────────────────────────────── Longitud total = 0x0054 = 84
|  |  └─────────────────────────────────── DSCP=0, ECN=0  (byte TOS = 0x00)
|  └────────────────────────────────────── IHL = 5
└───────────────────────────────────────── Versión = 4
```

## 1.3 El campo IHL y por qué la cabecera es de tamaño variable

La cabecera **no siempre son 20 bytes**. El campo **IHL** dice cuántos bloques de 4 bytes ocupa, y si hay **opciones**, ocupa más.

```
IHL = 5  →  5 × 4 = 20 bytes   (cabecera mínima, SIN opciones)
IHL = 6  →  6 × 4 = 24 bytes   (4 bytes = UNA opción de 4 bytes)
IHL = 15 → 15 × 4 = 60 bytes   (máximo: 40 bytes de opciones)
```

**Opciones.** Cada opción tiene el formato `1 byte tipo | 1 byte longitud | datos`. Los tipos históricos (de `RFC 791`) incluyen:

| Opción | Tipo | Qué hace |
|---|---|---|
| **RR** (*Record Route*) | 7 | **Freeze routing**: obliga a que el paquete pase por una lista de routers que escribe la fuente. Obsoleta, pero es la que da nombre a la opción. |
| **SSM** (*Source Route*) | 137 | El campo de destino se usa como **lista de routers por los que tiene que pasar**. Obsoleta y peligrosa. |
| **Timestamp** | 68 | Cada router que pasa mete la hora. Obsoleta. |
| **EOL** (*End of Option List*) | 0 | Marca de fin de las opciones. |
| **Pad** | 1 | Relleno para alinear a 4 bytes. |

> ⚠️ **Nota para 2026:** las opciones de IP (source route, record route, timestamp, y las loosely *Loose Source and Record Route* de traceroute) están **obsoletec. Se parsean pero no se usan**, y muchos routers y middleboxes las descartan o rechazan por motivos de seguridad. **En la práctica, IHL es siempre 5.** Decir "IHL = 5, 20 bytes, sin opciones" es la respuesta correcta en cualquier examen salvo que pregunten explícitamente por fragmentación o por options de seguridad.

## 1.4 La máscara de subred, aplicada bit a bit

La máscara es un número de 32 bits con **unos al principio y ceros al final**. El prefijo `/n` es la **cantidad de unos**. La operación que define todo es un **AND** bit a bit entre la dirección y la máscara:

```
dirección AND máscara = DIRECCIÓN DE RED
```

**Ejemplo completo: `192.168.10.55` con máscara `/26`.**

*Paso 1 — Escribo la dirección en binario, octeto por octeto.*

```
192 = 11000000    (128 + 64)
168 = 10101000    (128 + 32 + 8)
 10 = 00001010    (8 + 2)
 55 = 00110111    (32 + 16 + 4 + 2 + 1)

192.168.10.55 = 11000000.10101000.00001010.00110111
```

*Paso 2 — Escribo la máscara `/26` en binario y en decimal.*

```
/26  →  26 unos seguidos de 6 ceros
     →  11111111.11111111.11111111.11000000
     →  255.255.255.192      (128 + 64 = 192)
```

*Paso 3 — Hago el AND bit a bit.*

```
  dirección   11000000 . 10101000 . 00001010 . 00110111
  máscara     11111111 . 11111111 . 11111111 . 11000000
  ──────────────────────────────────────────────────────
  AND         11000000 . 10101000 . 00001010 . 00000000
              192      . 168      . 10       . 0
──────────────────────────────────────────────────────────
  AND        1&1 1&1 1&1 1&1 1&1 1&1 1&1 1&1
           = 1   1   1   1   1   1   1   1  → 11111111 = 255 ✓

  AND del 3er octeto: 00001010 AND 11111111 = 00001010 = 10 ✓

  AND del 4to octeto, que es donde pasa la separable:
    dirección  0 0 1 1 0 1 1 1
    máscara    1 1 0 0 0 0 0 0
    ──────────────────────────
    AND        0 0 0 0 0 0 0 0  = 0        ← los 2 unos de la máscara
                                             se apagaron
```

*Paso 4 — Interpreto el resultado.*

```
DIRECCIÓN DE RED = 192.168.10.0        y el prefijo es /26

Los bits de HOST de la dirección eran los 6 últimos:
  00110111  →  bits de red (los 2 primeros) = 00
               bits de host (los 6 últimos)  = 110111 = 55

Como 55 < 64 (= 2⁶), esta dirección cae en el PRIMER /26 de la /24.
```

*Paso 5 — Calculo el tamaño del bloque y los otros tres valores.*

```
Tamaño del bloque /26 = 2⁶ = 64 direcciones

DIRECCIÓN DE RED      = 192.168.10.0     (el AND de arriba)
RANGO DE HOSTS        = 192.168.10.1  a  192.168.10.62
                          (red + 1   hasta  red + 64 − 2)
BROADCAST             = 192.168.10.63    (red + 64 − 1)
NÚMERO DE HOSTS ÚTILES = 64 − 2 = 62     (2⁶ − 2)

Chequeo: 1 (red) + 62 (hosts) + 1 (broadcast) = 64 ✓
Chequeo: .0 + .63 = 64 direcciones numeradas, la última es 63 ✓
Chequeo: .55 está entre .1 y .62  ✓  (y es el host número 55)
```

*Paso 6 — Dibujo el /24 partido en cuatro /26 para verlo.*

```
192.168.10.0/24                    (256 direcciones)

.0                   .63   .64                 .127  .128          .191  .192       .255
┌──────────┬──────────┬──────────┬──────────┬──────────┬──────────┬──────────┬──────────┐
│ .0/26    │ .64/26   │ .128/26  │ .192/26  │ ↑ ESTA   │          │          │          │
│ red .0   │ red .64  │ red .128 │ red .192 │ 192.168.10.55/26    │          │          │
│ brdc .63 │ brdc .127│ brdc .191│ brdc .255│  = .0/26 │          │          │          │
└──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──────────┘
← 64 dir →  ← 64 →    ← 64 →     ← 64 →
```

**Y el error conceptual más común:** el **broadcast de la subred no es** `192.168.10.255`, es `192.168.10.63`. El broadcast depende de **la máscara**, no de la dirección. Con la `/24` el broadcast era `.255`; con la `/26` es `.63`. Esto se desarrolla entero en la Clase 27.

## 1.5 Direcciones claseful (A, B, C, D, E) y por qué quedaron obsoletas

Hasta 1993, una dirección IPv4 se leía **con la máscara fija según el primer bit**, sin necesidad de que nadie configurara nada. La tabla que hay que saber de memoria:

| Clase | Primer octeto en binario | Rango | Prefijo | Redes | Hosts por red | Uso |
|---|---|---|---|---|---|---|
| **A** | `0xxxxxxx` | 1 – 126 | **/8** | 126 | **16 777 214** | redes enormes (gobiernos, Multicast) |
| **B** | `10xxxxxx` | 128 – 191 | **/16** | 16 384 | **65 534** | redes grandes (universidades, empresas) |
| **C** | `110xxxxx` | 192 – 223 | **/24** | 2 097 152 | **254** | redes chicas (empresas, departamentos) |
| **D** | `1110xxxx` | 224 – 239 | **/4** | — | — | **multicast** |
| **E** | `1111xxxx` | 240 – 255 | **/4** | — | — | **reservada** para experimentación futura |

*Cómo se lee la clase de una dirección, sin ninguna máscara:* mirá el primer octeto en binario y fijate los primeros bits. `192 = 11000000` → empieza con `110` → **clase C** → máscara `/24`. `172 = 10101100` → empieza con `10` → **clase B** → `/16`. `10 = 00001010` → empieza con `0` → **clase A** → `/8`. `224 = 11100000` → `1110` → **clase D**.

**Por qué quedaron obsoletas — el cálculo que lo mata.** El problema era que **las clases no dejaban espacio intermedio**: o te sobraban direcciones (**clase B**) o te faltaban (**clase C**).

```
Una empresa con 300 hosts en 1993:

Necesita 300 direcciones.
Una clase C tiene 254   →  NO LE ALCANZA (300 > 254)
Una clase B tiene 65 534  →  LE SOBRA INFUMAMENTE

Con la clase B:
    direcciones totales          65 536
    utilizables (2¹⁶ − 2)        65 534
    usadas por la empresa             300
    ──────────────────────────────────────
    DESPERDICIADAS                65 234

O sea: el 99,5 % del bloque queda inútil. Y el agregado es
65 234, que es del orden de las decenas de miles (mucho más
que 22 000): por cada empresa con 300 hosts "compra" 65 000
direcciones que jamás va a usar.
```

El otro problema era simétrico: una biblioteca con **20** computers necesitaba una clase C (254) y desperdiciaba 234. Y el mercado lo notaba: **las clases B se agotaron primero** (las asignación del mundo entero eran sólo 16 384), mientras que las clase C sobraban. **Cualquiera podía pedir una clase B y agotarlas.** Ese fue el problema económico que forzó el cambio.

## La historia en una línea de tiempo

| Año | Hecho |
|---|---|
| 1981 | `RFC 791` define IPv4 **con clases**: A, B, C, más D (multicast) y E (reservada). |
| 1984 | `RFC 950` ("Internet Subnetting") introduce el **subnetting dentro de una clase**: una clase B prestada se divide en subredes. Esto obligaba a que **todos los routers de la red** usaran la misma máscara. |
| 1993 | **Fin de la asignación por clases.** Se implementa **CIDR** y la IANA empieza a asignar por **prefijo de cualquier longitud** ( `RFC 4632` ). Se introduce además el **routing CIDR** en los routers de frontera. |
| 1994 | El problema de las clases B se había resuelto **con subnetting**, que era una solución obligatoria y rígida; con CIDR la longitud del prefijo pasa a ser **una decisión local de cada enlace**. |
| 1996 | `RFC 1918` (verificar) define las **direcciones privadas** `10/8` , `172.16/12` y `192.168/16` : reutilizables internamente, **no enrutables en Internet**. |
| 2011 | **APNIC se agota**: es el último RIR que asigna direcciones IPv4 de clase C por la vía clásica. |
| 2019 | **RIPE NCC hace su última asignación** `/22` (25 de noviembre): Europa agota IPv4. ARIN ya se había agotado en 2015, LACNIC en 2020 y AfriNIC en 2017. |
| 2026 | **IPv4 es un recurso scarcity_planificado:** CGNAT en todos los ISPs y despliegue de IPv6 por encima del 50 % en los usuarios de Google. |

**El `/n` como atajo y el "número mágico".** Con CIDR, cualquier prefijo es legal, así que los números que importan son los que van del 32 al 0. La forma rápida de WORK es mirar el **"número mágico"**: el **número de hosts** de cada prefijo, que es `2^(32−n) − 2`. La tabla completa es el Anexo de la **`RFC 1878`** (***Requirements for IP Address Reservation*, 1996)**, y es la tabla que hay que tener a la vista cuando estás haciendo subnetting en un examen de papel:

```
/n    hosts      /n    hosts      /n    hosts      /n    hosts
────  ───────    ────  ───────    ────  ───────    ────  ───────
/32      1       /24    254       /17   32 766     /10   1 048 574
/31      0 (*)   /23    510       /16   65 534     /9    2 097 150
/30      2       /22  1 022       /15  131 070     /8    4 194 302
/29      6       /21  2 046       /14  262 142     /7    8 388 606
/28     14       /20  4 094       /13  524 286     /6   16 777 214
/27     30       /19  8 190       /12  1 048 574   /5   33 554 430
/26     62       /18 16 382       /11  2 097 150   /4   67 108 862
/25    126       ...   ...        ...   ...        ...   ...      ...

(*) el /31 tiene 2 direcciones y 0 hosts "útiles" en el sentido
    clásico; RFC 3021 (verificar) lo legitima para enlaces
    punto a punto.  /32 = una sola dirección, el loopback.

Para leerlo al revés (esto es el atajo del subnetting):
  "necesito 100 hosts"  →  el menor número ≥ 100 de la tabla es 126 → /25
  "necesito 200 hosts"  →  254 → /24
  "necesito 50 hosts"   →  62 → /26
  "necesito 20 hosts"   →  30 → /27
  "necesito 13 hosts"   →  14 → /28
```

## 1.6 CIDR: por qué existe

**CIDR = Classless Inter-Domain Routing** ( `RFC 4632` , 2006). Tres cambios, y los tres importan:

1. **La longitud del prefijo es libre.** Ya no hay clases: `/8` , `/13` , `/27` , cualquier valor entre 0 y 32. Se elimina por completo la frontera entre clases.
2. **La máscara es local, no global.** Antes del subnetting, todos los routers de la red tenían que usar **la misma** máscara. Con CIDR, cada enlace puede usar la que necesite, y un router puede tener **máscaras distintas en cada interfaz** (es exactamente lo que se ve en las tablas de enrutamiento: `192.168.1.0/24` en una línea y `220.10.10.28/30` en otra, en el mismo router).
3. **La sumarización (agregación) vuelve a ser posible.** Un router que se ocupa de `192.168.0.0/16` puede anunciar **un solo prefijo** en vez de 256. Eso reduce las tablas de enrutamiento de toda la Internet deBackbone. Sin CIDR, la tabla de un router de núcleo habría tenido que tener una entrada por cada red de cada organización del mundo.

**La consecuencia concreta en un examen.** Cuando el enunciado dice "asigne una red para 300 hosts", **la respuesta de hoy no es "clase B"**: es "el menor prefijo que dé 300" → `/23` (510 hosts). Y cuando dice "asigne una red y quiere dejar espacio para crecer", la respuesta es un bloque `/22` o `/21` **y después subnetting** (Clase 27 y 28). **Nadie dice "clase B" en 2026.**

## 1.7 Direcciones especiales

| Rango | Nombre | Qué es y para qué está |
|---|---|---|
| `0.0.0.0/8` | "Cero" / *"this host on this network"* | En un **servidor** significa *"escuchá en todas las interfaces"* ( `bind(0.0.0.0)` ). En una **tabla de enrutamiento** es la **ruta por defecto** ( `0.0.0.0/0` , todos los destinos). Como **dirección de origen**, significa *"no tengo dirección"* — es lo que manda un host antes de tener IP por DHCP ( `DHCPDISCOVER` ). **En ninguna de las tres es una dirección de un host.** |
| `10.0.0.0/8` `172.16.0.0/12` `192.168.0.0/16` | **Privadas** ( `RFC 1918` , verificar) | **Se pueden repetir en todas las redes del mundo** porque **no se enrutan en Internet**: ningún router de backbone tiene una ruta hacia ellas. Existen porque **IPv4 se agotó** y hace falta reutilizar el mismo espacio internamente. Se usan detrás de un **NAT** (Clase 31). **10/8** = 16 777 214 direcciones · **172.16/12** = 1 048 574 · **192.168/16** = 65 534. |
| `127.0.0.0/8` | **Loopback** | El **propio host**. Todo lo que se mande a `127.x.x.x` lo entrega el kernel localmente, sin salir a la red. `127.0.0.1` se llama *localhost*. Sirve para probar un servicio en la misma máquina sin tocar la red. |
| `169.254.0.0/16` | **Link-local** (*APIPA*) | **Se autoasigna el host cuando el DHCP falla.** El host elige una dirección al azar en ese rango y se verifica con ARP (DAD). Sirve para comunicarse **en el enlace local**, pero **no se enruta**: nunca pasa de la LAN. Es el rango de las cámaras IP mal configuradas y de los enlaces punto a punto sin DHCP. La definición formal está en la `RFC 3927` (verificar). |
| `224.0.0.0/4` | **Multicast** (clase D) | Grupo de destinos. **La dirección es del grupo, no del host.** `224.0.0.0` a `224.0.0.255` es *link-local* (IGMP, `224.0.0.5` y `.6` son los multicast de OSPF); `239.0.0.0/8` es *administratively scoped* (globales, como los de Cisco Discovery Protocol). El prefijo `/4` es lo que dice la clase D, **no** una máscara que se configure. |
| `240.0.0.0/4` | Reservada (clase E) | Reservada para uso futuro. En la práctica el último octeto de `255` se usa para **broadcast**, lo que pisa parcialmente el rango. |
| `255.255.255.255` | **Broadcast limitado** | Broadcast **de la LAN local, y sólo de ella**. Los routers **no lo reenvían**, jamás. Es el `destino` del ARP request, del DHCP Discover y de los announcing gratuitous ARP. |
| `100.64.0.0/10` | *Shared Address Space* (CGNAT) | El rango que usan los operadores para la **traducción de direcciones deCarrier-Grade NAT** ( `RFC 6598` , verificar). No es privado ni público: es "compartido", por eso sólo se enruta dentro del operador. |

**Por qué existen las privadas, en una frase.** Porque el espacio de IPv4 se **agotó** y porque la topología de Internet es **jerárquica**: no todo lo que hay en el mundo tiene que ser alcanzable desde todo el mundo. Si tu notebook tiene `192.168.1.5` y el servidor del otro lado del mundo también tiene `192.168.1.5` , no hay ningún conflicto **porque ningún router de Internet enruta ese rango**. La reutilización es lo que nos salvó de haber agotado IPv4 en 1994.

## 1.8 ⚠️ Actualización a 2026

**Sigue igual, de memoria:**

- El **formato de la cabecera IPv4**: 20 bytes mínimos, IHL × 4, los mismos 13 campos, el mismo orden. **No cambió desde 1981** y no va a cambiar.
- **Máscara = unos al principio, ceros al final**; dirección de red = AND; broadcast = red + tamaño − 1.
- Las **clases A–E** (historia), el **/n** y el **número mágico**.
- Los **rangos especiales**: `0/8`, `10/8`, `172.16/12`, `127/8`, `169.254/16`, `192.168/16`, `224/4`, `255.255.255.255`.

**Cambió:**

- **La claseful es historia, y hace 30 años que lo es.** Desde 1993 la asignación es por prefijo ( `RFC 4632` ) y **las clases no significan nada en el mundo real**: el `192.168.1.0/24` de tu casa no es "una clase C" sino un `/24`. Se enseñan porque (a) aparecen en los exámenes viejos, (b) explican por qué existe CIDR, y (c) la máscara `/24` **se parece** a una clase C por casualidad histórica. ⇒ **En un examen de hoy, nunca respondas "clase C" como solución; respondá "/24".** Si el enunciado dice "clase C", traducilo a `/24` y seguí.
- **IPv4 se agotó, de verdad y por etapas.** APNIC se agotó en abril de 2011, ARIN en septiembre de 2015, AfriNIC en 2017, **RIPE NCC hizo su última asignación `/22` el 25 de noviembre de 2019**, y LACNIC en 2020. Hoy RIPE mantiene una *waiting list* para un único `/24` recuperado. ⇒ **Consejo de examen:** si te preguntan "¿por qué existen las direcciones privadas?", la respuesta es literalmente "porque IPv4 se agotó", y **la cifra para mencionar es la del 25-nov-2019 de RIPE**.
- **CGNAT es el nuevo default de la capa 3 de acceso.** `100.64.0.0/10` es el rango que el operador traduce, y eso rompe conectividad entrante, complica la señalización SIP/IMS y rompe IPsec y peer-to-peer. **Es la respuesta moderna a "mi dirección es privada"**: antes lala traducción era opcional (NAT en la empresa), ahora es la norma en el acceso.
- **IPv6 pasó la mitad.** El **28 de marzo de 2026** el acceso a Google por IPv6 superó el **50 %** (50,10 %) por primera vez. Pero es desigual: Francia e India por encima del 70 %, China por debajo del 5 %. ⇒ **Consecuencia para el examen:** si te preguntan "¿IPv6 reemplazó a IPv4?", la respuesta es "a medias y por país"; lo que se mide es distinto en cada fuente, así que **declará qué estás midiendo**.
- **La cabecera IPv6 es fija: 40 bytes, sin IHL, sin opciones, sin fragmentación en los routers.** Los routers IPv6 **no** fragmentan; fragmenta el origen. El concepto de "opciones de cabecera IP" no existe en IPv6: lo que hay son **cabeceras de extensión** encadenadas por el campo *Next Header*, que **no** las ve ningún router intermedio salvo la de *Hop-by-Hop* ( `RFC 8200` ). ⇒ El campo **IHL y todo el mecanismo de opciones de IPv4 es `legacy`** y sólo se estudia por historia.
- **Seguridad.** El **source routing de IPv4 sigue siendo un vector de ataque** y muchos middleboxes lo bloquean. `RFC 7126` (verificar) habla del problema de la cabecera IPv4 con IHL; no lo uses si no lo verificaste.

**Consecuencia para el examen:** la cabecera IPv4 se sigue preguntando **exactamente igual**. Lo que hay que sumar es una línea: "*IPv4 está agotado desde 2019 (RIPE), CGNAT es el default del acceso, y el /n con el número mágico reemplazó a las clases; las clases sólo se explican como historia.*"

---

# PARTE 2 — PRÁCTICA RESUELTA

## 2.1 Ejercicio 1 — Binario y máscara, de punta a punta

**Datos.** La dirección `172.31.199.14` con máscara `/22` (máscara de red y de la interfaz del router).

**Resolución paso a paso.**

*Paso 1 — La dirección a binario.*

```
172 = 10101100    128 + 32 + 8 + 4
 31 = 00011111    16 + 8 + 4 + 2 + 1
199 = 11000111    128 + 64 + 4 + 2 + 1
 14 = 00001110    8 + 4 + 2
```

*Paso 2 — La máscara `/22`.*

```
/22  →  22 unos + 10 ceros
     →  11111111 . 11111111 . 11111100 . 00000000
     →   255      . 255      . 252      . 0        = 255.255.252.0

Chequeo: 255 + 255 + 252 + 0 = 762. Y 3×255 = 765, así que
         765 − 762 = 3  →  hay 3 ceros en los dos últimos octetos.
         8 + 8 + 6 + 0 = 22  ✓  (el /22 tiene 22 unos)
```

*Paso 3 — El AND, octeto por octeto.*

```
octeto      dirección   máscara     AND
──────      ─────────   ────────    ───
  1°         10101100    11111111    10101100  = 172
  2°         00011111    11111111    00011111  =  31
  3°         11000111    11111100    11000100  = 196   ← acá cambia
  4°         00001110    00000000    00000000  =   0   ← y acá se apaga

DIRECCIÓN DE RED = 172.31.196.0/22

Ojo con el 3° octeto: 199 AND 252 = 196, NO 199.
Los 6 unos de la máscara "comen" los 2 bits más bajos de 199.
199 = 11000111
      11111100
AND = 11000100 = 196
```

*Paso 4 — Tamaño del bloque, hosts, broadcast.*

```
Tamaño = 2^(32-22) = 2¹⁰ = 1024 direcciones
Hosts útiles = 1024 - 2 = 1022

RED      = 172.31.196.0
PRIMER   = 172.31.196.1        (red + 1)
ÚLTIMO   = 172.31.199.254      (red + 1024 - 2)
BROADCAST= 172.31.199.255      (red + 1024 - 1)

Chequeo: 172.31.199.14 está entre .196.1 y .199.254  ✓
¿Es broadcast? No (sólo lo es .199.255)  ✓
¿Es la red? No (la red es .196.0)  ✓
```

*Paso 5 — El detalle fino: el número de host.*

```
Bits de host = los 10 últimos (todo el 4° octeto + 2 del 3°)

172.31.199.14
           3° octeto: 199 = 11000111 → los 2 bits de host son "11" = 3
           4° octeto:  14 = 00001110 = 14

número de host = 3 × 256 + 14 = 768 + 14 = 782
(2 bits de host del 3er octeto ⇒ 2² = 4 valores posibles, de 0 a 3)
```

*Interpretación:* esta dirección está en la **cuarta y última** `/22` de la `/20` `172.31.192.0/20`. Los bloques `/22` de esa `/20` son `.192.0`, `.196.0`, `.200.0` y `.204.0`. Si en un ejercicio te piden "dividir `172.31.192.0/20` en cuatro `/22`", la respuesta es exactamente esa cuaterna.

## 2.2 Ejercicio 2 — Clase, máscara por defecto y capacidad

**Datos.** Determinar la clase y la máscara por defecto de estas cinco direcciones, y cuántas direcciones hosts útiles admite cada red.

| Dirección | Clase | ¿Claseful o CIDR? | Máscara por defecto | Hosts útiles | Nº de redes de ese tamaño |
|---|---|---|---|---|---|
| `10.4.3.2` | **A** ( `00001010` , empieza con `0` ) | claseful | `/8` = 255.0.0.0 | $2^{24} - 2 =$ **16 777 214** | $2^{7} - 2 =$ **126** |
| `172.16.40.9` | **B** ( `10101100` , empieza con `10` ) | claseful | `/16` = 255.255.0.0 | $2^{16} - 2 =$ **65 534** | $2^{14} =$ **16 384** |
| `192.168.1.1` | **C** ( `11000000` , empieza con `110` ) | claseful | `/24` = 255.255.255.0 | $2^{8} - 2 =$ **254** | $2^{21} =$ **2 097 152** |
| `224.0.0.251` | **D** ( `11100000` , empieza con `1110` ) | multicast | `/4` (no se configura) | — | — |
| `240.1.2.3` | **E** ( `11110000` , empieza con `1111` ) | reservada | `/4` (no se configura) | — | — |

**La cuenta de cada fila, a la vista:**

```
A  →  /8   → 32−8 = 24 bits de host → 2²⁴ − 2 = 16 777 214
B  →  /16  → 32−16 = 16 bits de host → 2¹⁶ − 2 = 65 534
C  →  /24  → 32−24 = 8 bits de host  → 2⁸  − 2 = 254
D  →  multicast: la dirección es del GRUPO, no de un host
E  →  reservada: no se asigna
```

**Las dos observaciones que hay que hacer en un examen:**

1. **La clase se lee del primer octeto, sin máscara.** Es un residuo histórico: los routers antigos lo usaban para deducir la máscara. **Hoy es inútil**: la clase de `10.4.3.2` y de `10.0.0.1` es la misma (A), pero un `/8` y un `/16` pueden tener la misma clase.
2. **Diez es clase A, y sin embargo `10.0.0.0/8` es una dirección privada.** Las privadas no se definieron por la clase sino por rangos fijos dentro de ella. Esta confusión es la que hace que los alumnos digan "clase A privada": lo correcto es "rango `10/8`, que está dentro de la clase A por su primer bit, pero que es privado por definición del RFC, no por su clase".

## 2.3 Ejercicio 3 — El checksum de la cabecera IPv4, cálculo completo

**Datos.** Un datagrama con estos campos:

```
Versión = 4         IHL = 5          DSCP = 0, ECN = 0
Longitud total      = 60 bytes (0x003C)
Identificación      = 0x1C46
Flags/Offset        = 0x4000   (DF = 1, MF = 0, offset = 0)
TTL                 = 64 (0x40)
Protocolo           = 6 (TCP)
Checksum            = 0x0000   ← hay que calcularlo
Origen              = 192.168.10.55  (0xC0A80A37)
Destino             = 219.41.31.21  (0xDB291F15)
```

**Resolución: se suman las mitades de 16 bits, se sigue el "acarreo", y se toma el complemento.**

```
La cabecera son 10 palabras de 16 bits (20 bytes ÷ 2):

 1)  4500     Versión+IHL = 0x45,   DSCP+ECN = 0x00
 2)  003C     Longitud total = 60
 3)  1C46     Identificación
 4)  4000     Flags = 100, offset = 0
 5)  4006     TTL = 0x40, Protocolo = 0x06
 6)  0000     Checksum (por ahora cero)
 7)  C0A8     Origen, 1ª mitad
 8)  0A37     Origen, 2ª mitad   (0x0A = 10, 0x37 = 55)
 9)  DB29     Destino, 1ª mitad  (0xDB = 219, 0x29 = 41)
10)  1F15     Destino, 2ª mitad  (0x1F = 31, 0x15 = 21)

SUMA (con acarreo: si da 5 dígitos hex, se suma el acarreo y se
deja el resto):

 4500 + 003C = 453C
 453C + 1C46 = 6182
 6182 + 4000 = A182
 A182 + 4006 = E188
 E188 + 0000 = E188
 E188 + C0A8 = 1A230   →  acarreo 1  →  A231        ← ¡primer acarreo!
 A231 + 0A37 = AC68
 AC68 + DB29 = 17768   →  acarreo 1  →  8792       ← ¡segundo acarreo!
 8792 + 1F15 = A6A7

CHECKSUM = complemento a uno de A6A7
         = NOT(A6A7) = 5958

La cabecera completa queda:
   45 00 00 3C 1C46 4000 40 06 59 58 C0A8 0A37 DB29 1F15
```

**Verificación (siempre hay que hacerla, y en el examen suma puntos):**

```
Volvemos a sumar TODO, esta vez con el checksum ya puesto:

 (4500 + 003C + 1C46 + 4000 + 4006 + 0000 + C0A8 + 0A37 + DB29 + 1F15)
                                                    =  A6A7
 A6A7 + 5958 =  FFFF

Si la suma da 0xFFFF, el checksum es correcto.
Esa es la definición de "complemento a uno": el checksum es el
valor que hace que la verificación dé todo 1s.
```

*Interpretación:* el checksum **cubre sólo la cabecera**, y por eso **no se recalcula en los routers** (que sólo cambian TTL, y ese campo tiene su propio mecanismo: el checksum se ajusta en incremental). El del **TCP y el del UDP** cubren cabecera **y** datos, con una pseudo-cabecera de IP que incluye las direcciones; por eso un NAT que cambia una IP **tiene que recalcular el checksum del transporte**.

## 2.4 Ejercicio 4 — ¿Por qué DHCP asigna `169.254.x.x` y no `0.0.0.0`?

**Enunciado.** Cuando un host busca un servidor DHCP y no encuentra ninguno, se autoasigna una dirección del rango `169.254.0.0/16` (APIPA). ¿Por qué **no** puede autoasignarse `0.0.0.0`? Y si puede usar `169.254.x.x`, ¿por qué no se usa `192.168.1.5` "porque total es mi red"?

**Resolución — la regla general primero: los rangos especiales están definidos por lo que hacen con ellos.**

| Rango | Semántica | ¿Se puede autoasignar un host? |
|---|---|---|
| `0.0.0.0/8` | "*no tengo dirección*" (fuente) · "*escuchá en todas*" (servidor) · "*por cualquier lado*" (ruta por defecto) | **No.** Significa **inexistencia** de dirección. Si un host se autoasignara `0.0.0.0`, los demás no sabrían ni si siquiera que hay alguien, y no podría arrancar ninguna conexión: **no hay dirección a la que responder.** |
| `127.0.0.0/8` | "*yo mismo*" | **No.** Está cableada al loopback del kernel, así que `127.x.x.x` **nunca sale a la red**: sería inútil para comunicarse con los demás. |
| `169.254.0.0/16` | "*este enlace local*" | **Sí.** Es un rango **asignado justamente para la autoconfiguración**: el host elige una dirección al azar de 16 bits, manda un **ARP request** con esa dirección (**DAD**, *Duplicate Address Detection*, ver `RFC 3927`, verificar) y si nadie contesta, se la queda. |
| `10/8`, `172.16/12`, `192.168/16` | "*privada*" | **No se puede adivinar.** Una dirección privada es sólo válida **detrás de un router que hace NAT**, y ese router tiene que tener una **tabla de traducción** con una entrada para esa dirección. Si el host se autoasigna `192.168.1.5` sin NAT, **el router no tiene entrada y descarta el paquete de salida.** |
| `224.0.0.0/4` | "*grupo*" | **No.** La dirección es del **grupo**, no del emisor: la fuente también necesita una dirección unicast. |

**Por qué `169.254/16` sí funciona, en tres pasos.**

```
t0   El host arranca. No tiene IP. Configura su IPv4 como
     0.0.0.0 (significa "ninguna") y la máscara a 0.0.0.0.

t1   Intenta DHCP. No hay servidor → timeout.

t2   El host se AUTOASIGNA una dirección al azar de 169.254/16
     (en Windows, además elige un sufijo de fecha y hora).
     Toma la máscara /16, que viene de la definición del rango:
         169.254.0.0  169.254.255.255
         ↑               ↑
         169 = 10101001  se fija el /16
         254 = 11111110

t3   Manda un ARP request por esa dirección para hacer DAD
     ("¿hay alguien con esta dirección?"). Si nadie contesta
     en un par de segundos, se la queda.

t4   A partir de ahí puede hablar con los demás del ENLACE, y
     sólo del enlace: 169.254/16 es link-local, los routers
     NO lo enrutan. Sirve para compartir archivos dentro de la red local, y nada más.
```

**La respuesta a "¿por qué no `192.168.1.5`?", que es la parte interesante:** porque `192.168/16` **existe sólo dentro de una red que alguien administrationó**. La dirección privada es una **convención entre el administrador y el NAT**, no una dirección "del mundo". Si el host se autoasignara una sin NAT detrás, el paquete saldría del host con destino/source `192.168.1.5`, **el router de salida lo descartaría** (no tiene

ruta ni entrada de traducción) y el host **quedaría sin comunicación**, que es exactamente el problema que se intenta resolver. `169.254/16` **está diseñado para ser usable sin ninguna infraestructura detrás**; `192.168/16` no.

> En clase: si te preguntan "qué hace el host si no tiene DHCP", la respuesta completa es: `0.0.0.0` → espera → se autoasigna `169.254.x.x` con **DAD** → habla **sólo en el enlace local** → si tiene un APIPA válido pero no gateway, la aplicación que necesita Internet **falla aunque el host "tenga IP"**. Ese último punto es la trampa de examen.

---

# PARTE 3 — PROBLEMAS DE EXAMEN DE FING

## 3.1 ⭐ Examen 2010-03, Problema 1 (30 puntos) — tres subredes /24, R1 y R2

> **Fuente:** Examen `eirc1003` / `sirc1003` — 24 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf **Texto del examen:**
>
> > Sea la red compuesta de tres subredes interconectadas por los routers R1 y R2 presentada en la siguiente figura. En la misma se indican los nodos conectados en cada subred así como la dirección IP de cada interfaz. Para su direccionamiento se asignó una red /24 a cada una de las subredes como se presenta en la figura, y se supone la existencia de 100 hosts en cada subred. El enrutamiento es estático. **a)** i. Indique las tablas de enrutamiento de los routers R1 y R2 de forma que permita conectividad extremo a extremo entre todos los equipos de la red. ii. Indique las tablas de enrutamiento para los hosts de cada subred. **b)** Se sustituye el router R2 por un switch, al que se conectan todos los equipos de las subredes 2 y 3. Para lograr un nuevo esquema de numeración operativo, se desea modificar la menor cantidad de direcciones IP posibles. i. Justifique la necesidad de renumerar alguna red para lograr una solución al problema. ii. Represente gráficamente su solución... iii. Indique la tabla de enrutamiento del router R1...

**Solución oficial (literal):**

> **a-i)** R1: `192.168.0.0/24 → eth0`, `192.168.1.0/24 → eth1`, `192.168.2.0/24 → next hop 192.168.1.2, eth1`. R2: `192.168.2.0/24 → eth1`, `192.168.1.0/24 → eth0`, `192.168.0.0/24 → next hop 192.168.1.1, eth0`. **a-ii)** Hosts de la subred 1: `192.168.0.0/24 → eth0`, `0.0.0.0 → 192.168.0.1`. Hosts subred 2: `192.168.1.0/24`, `0.0.0.0 → 192.168.1.1`. Hosts subred 3: `192.168.2.0/24`, `0.0.0.0 → 192.168.2.1`. **b-i)** *"Las direcciones IP consideradas para las subredes 1, 2 y 3 corresponden a redes contiguas. Para agrupar direcciones se debe modificar la máscara de red, disminuyendo la cantidad de bits de la máscara [...] El problema que surge es que para incluir en una subred las subredes 192.168.1.0/24 y 192.168.2.0/24 se necesita una máscara /22 que incluye la red 192.168.0.0/24. Por lo tanto la solución es renumerar tal que las subredes sean diferenciables por la máscara."* **b-ii)** Dos soluciones: renumerar la subred 1 como `192.168.4.0/24` (quedando subredes 2 y 3 como `192.168.0.0/22`), **o** renumerar la subred 2 como `192.168.3.0/23` (quedando `192.168.0.0/24` y `192.168.2.0/23`). *"De las soluciones propuestas la primera es la ideal dado que modifica una IP menos."* **b-iii)** R1: `192.168.4.0/24 → eth0`, `192.168.0.0/22 → eth1`.

### a-i) Tablas de enrutamiento de R1 y de R2

*Suposición que hago y justifico:* por el enunciado, cada subred es una `/24` y hay 100 hosts en cada una. Las direcciones de gateway (`192.168.0.1`, `192.168.1.1`, `192.168.1.2`, `192.168.2.1`) son las que da el enunciado en la figura.

**R1** — tiene una interfaz en la subred 1 (`eth0`) y otra en la subred 2 (`eth1`):

```
Configuración R1
Red destino / Máscara      Gateway (next hop)   Interfaz
─────────────────────────  ──────────────────   ─────────
192.168.0.0 / 255.255.255.0   –                 eth0
192.168.1.0 / 255.255.255.0   –                 eth1
192.168.2.0 / 255.255.255.0   192.168.1.2       eth1
```

**¿Por qué la tercera línea lleva next hop `192.168.1.2` y no `192.168.2.1`?** Porque `192.168.1.2` es la dirección **de la interfaz de R2 dentro de la red que R1 ya tiene conectada**. R1 no tiene por qué conocer la dirección de un host o de otra red: le alcanza con el vecino. **Y esa es la razón de existir del campo *Gateway* de una tabla de reenvío** (Clase 24, §1.1): el router necesita un next hop **alcanzable directamente**.

**R2** — una interfaz en la subred 2 (`eth0`) y otra en la subred 3 (`eth1`):

```
Configuración R2
Red destino / Máscara      Gateway (next hop)   Interfaz
─────────────────────────  ──────────────────   ─────────
192.168.2.0 / 255.255.255.0   –                 eth1
192.168.1.0 / 255.255.255.0   –                 eth0
192.168.0.0 / 255.255.255.0   192.168.1.1       eth0
```

*Interpretación:* **la fila "directamente conectada" no tiene next hop**, porque no hay nadie a quien saltar: el router entrega el paquete directo a la interfaz. Y la fila con next hop **también lleva interfaz**, porque el next hop dice *a qué router* y la interfaz dice *por qué cable*.

### a-ii) Tablas de enrutamiento de los hosts

```
Hosts subred 1                    Hosts subred 2
Red destino / Máscara   Gateway   Red destino / Máscara   Gateway
192.168.0.0 /24         –         192.168.1.0 /24         –
0.0.0.0     /0          192.168.0.1  0.0.0.0  /0        192.168.1.1

Hosts subred 3
Red destino / Máscara   Gateway
192.168.2.0 /24         –
0.0.0.0     /0          192.168.2.1
```

*Por qué el host tiene una sola entrada además de la red:* porque un host **no enruta**, sólo entrega. Todo lo que no sea de su propia subred va al **default gateway** (`0.0.0.0/0`). Y **por qué el gateway es el router que está en su propia subred** (`192.168.0.1` para la subred 1): por la misma razón del *next hop* de arriba: para poder hablar con el gateway tiene que poder **alcanzarlo directamente**, o sea que su IP debe estar en la subred del host.

*Suposición justificada:* los routers usan la **dirección más baja** de cada subred como gateway (`192.168.0.1`, `.1.1`, `.2.1`), que es la convención que aparece en las soluciones oficiales de FING y la que se usa en el enunciado (`sirc1003`).

### b-i) ¿Por qué hay que renumerar?

Acá está el corazón conceptual de la clase, y es **sumarización** aplicada bit a bit.

**Lo que se quiere:** un solo switch para las subredes 2 y 3 ⇒ **una sola subred IP** que las contenga a las dos. Si R1 tiene que announcement una sola entrada para las dos, ambas tienen que ser **contiguas en el espacio de direcciones** y tener un prefijo común que las cubra.

**Intentando incluir `192.168.1.0/24` y `192.168.2.0/24`:**

```
Bloque /22 que empieza en 192.168.0.0:
  192.168.0.0/22  =  192.168.0.0  a  192.168.3.255
  ▲              ▲
  incluye        incluye hasta 192.168.3.255
  192.168.0.0/24
  e incluye 192.168.1.0/24 y 192.168.2.0/24  ✓

¿Es el prefijo más corto que las cubre? Sí:
  /23 desde 192.168.0.0  = 192.168.0.0  a  192.168.1.255   → NO incluye la .2
  /23 desde 192.168.2.0  = 192.168.2.0  a  192.168.3.255   → NO incluye la .1

Entonces el mínimo es /22, y ese /22 mete adentro
192.168.0.0/24, que es LA SUBRED 1.  ← el problema
```

**Verificación bit a bit de por qué el /22 es el mínimo** (esto es lo que hay que saber hacer):

```
Los tres /24 de la subred:
  192.168.0.0/24   tercer octeto: 0   = 00000000
  192.168.1.0/24   tercer octeto: 1   = 00000001
  192.168.2.0/24   tercer octeto: 2   = 00000010
                                                ▲
  los 3 octetos son iguales → los bits que importan
  son los últimos 8 del TERCER octeto

Para que un prefijo cubra los valores 0, 1 y 2, tiene que
cubrir el rango 0 a 3, porque los bits que los distinguen son
los DOS últimos:

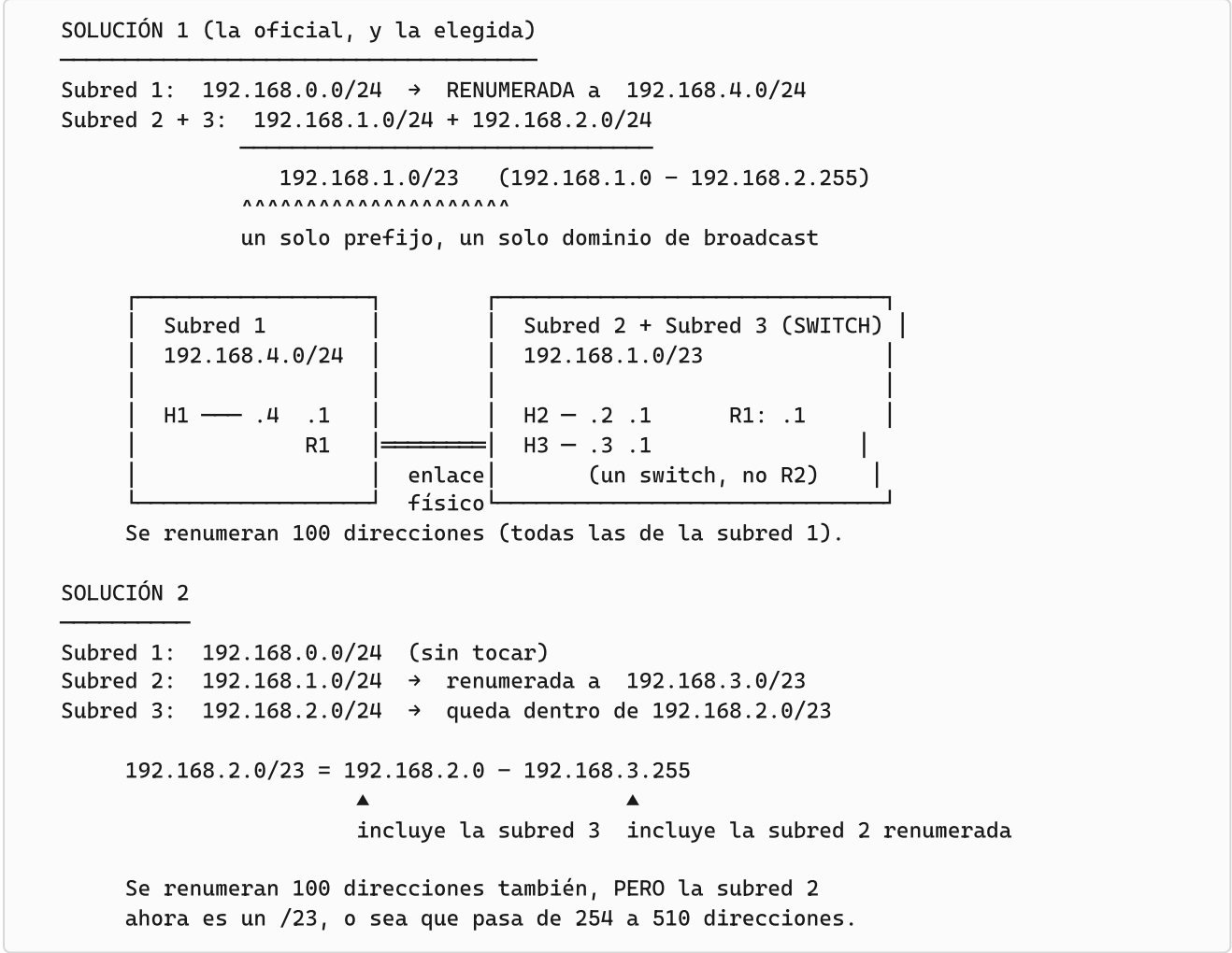
  00000000 = 0    ┐
  00000001 = 1    ├─ los dos últimos bits
  00000010 = 2    ┘

  Un prefijo que no toque esos 2 bits tiene que llegar hasta
  el 3er octeto = 00000011 (= 3)  →  /22  →  192.168.0.0/22

Un /23 sólo congelaría 1 bit (0000000_), o sea o el 0-1 o el
2-3, y por lo tanto no alcanza para los tres.

Conclusión: cualquier prefijo que cubra las subredes 1, 2 y 3
(en la numeración original) tiene que incluir 192.168.0.0/24.
⇒ hay que renumerar.
```

## b-ii) Las dos soluciones, y por qué la primera es la mejor




**Comparación honesta de las dos (esto es lo que el examen pide implícitamente con "la menor cantidad de direcciones posibles"):**

| | Solución 1 | Solución 2 |
|---|---|---|
| Subredes renumeradas | **1** (la subred 1) | **1** (la subred 2) |
| Direcciones de host modificadas | ~100 | ~100 |
| Tamaño del bloque del switch | `/23` = 510 direcciones | `/23` = 510 direcciones |
| Direcciones de **red** de las subredes | 1 renumerada | 1 renumerada |
| **Problema de la 2** | — | la subred 2 pasa de `/24` a `/23`: **cambia su máscara**, o sea que hay que reconfigurar la máscara de 100 hosts además de la dirección |
| Tabla de R1 | 2 entradas | 2 entradas |

**La oficial prefiere la 1** porque en la 1 **sólo cambian direcciones IP**: la subred renumerada conserva su `/24`, así que su máscara no cambia y los hosts sólo cambian su IP. En la 2 hay que cambiar IP **y** máscara. Con una red de 100 hosts eso es la misma cantidad de direcciones, pero **más configuración**.

*Suposición que hago y declaro:* el enunciado dice "modificar la menor cantidad de **direcciones IP** posibles", y en sentido estricto las dos soluciones modifican un número parecido de direcciones. Si en tu material aparece sólo la solución 1, es por este criterio de máscara. **Si el enunciado te permitiera dejar una subred con máscara /23 y no te importara la reconfiguración, la 2 también es válida** — y eso es exactamente el tipo de decisión que hay que justificar por escrito en un examen.

### b-iii) La nueva tabla de R1 (solución 1)

```
Configuración R1
Red destino / Máscara      Gateway   Interfaz
─────────────────────────  ────────  ─────────
192.168.4.0 / 255.255.255.0   –      eth0
192.168.0.0 / 255.255.255.0   –      eth1     ← 192.168.0.0/22
192.168.1.0 / 255.255.254.0   –      eth1
```

*Interpretación:* la subred 1 **dejó de estar en** `192.168.0.0/24` y pasó a `192.168.4.0/24`, así que R1 ya **no** necesita una entrada para ella: ahora la anuncia con el **default** `0.0.0.0/0` hacia su ISP, que es lo que corresponde a una red privada. Y la subred del switch cambió de máscara de `/24` a `/23`: los hosts de la subred 3 (`192.168.2.x`) ahora son de la **misma subred** que los de la subred 2, así que **ya no necesitan salir al router para comunicarse entre ellos**: se entregan directo por el switch. **Ese es el beneficio real del ejercicio**: menos tránsito por el router, menos encapsulación, y un dominio de broadcast compartido.

## 3.2 Nota: cuándo sí se pueden agrupar dos `/24` — el caso del Examen 2009-12

El `sirc0912` Problema 1 usa las subredes `192.168.108.0/24`, `192.168.109.0/24` y `192.168.111.0/24`, y su solución oficial **agrupa 108 y 109 en** `192.168.108.0/23` sin renumerar nada. ¿Por qué ahí sí y en el caso anterior no? **Por la paridad del tercer octeto**, y es el mismo razonamiento bit a bit:

```
/23 = el último bit del TERCER octeto queda libre.
Un /23 cubre X.0.0 – X.1.255, es decir los pares {0,1}, {2,3},
{4,5}, ... Necesita que el 3er octeto sea PAR para arrancar.

108 = 01101100  (par)   109 = 01101101  (impar)
     └─ los 6 primeros bits iguales, difieren sólo en el último
→ 01101100|0 = 192.168.108.0/23  los cubre a los dos   ✓ SIN renumerar

111 = 01101111  (impar)  110 = 01101110  (par)
→ 0110111|0 = 111 = 192.168.110.0/23  cubre 110 y 111   ✓
→ y 192.168.108.0/23 cubre 108 y 109                      ✓

Pero 110, 111, 112, 113 sería 4 redes y ningún /23 (ni /22
alcanzable sin arrastrar la 192.168.108.0/24) las cubre todas
sin meter una subred de más.
```

**La regla para el examen, en una línea:** *dos* `/24` *contiguos se pueden sumar en un* `/23` *si y sólo si el primero tiene el tercer octeto par.* Si es impar, hay que renumerar. Y para tres `/24` hay que mirar los **dos** últimos bits, como hicimos en §3.1 b-i.

---

# PARTE 4 — CIERRE

## 📌 Resumen

1. La cabecera IPv4 mínima son **20 bytes** = **5 filas de 32 bits**, con **13 campos**. **No cambió desde 1981.**
2. **IHL** dice el tamaño de la cabecera en **palabras de 4 bytes**: `IHL × 4`. Con opciones, de 20 a 60 bytes. **En la práctica, IHL = 5 siempre.**
3. **Dirección de red = dirección AND máscara. Broadcast = red + 2^(32−n) − 1. Rango de hosts = red + 1 … red + 2^(32−n) − 2. Chequeo: `2^(32−n) − 2` hosts útiles.**
4. Las **clases A/B/C/D/E** son historia de 1981. Con una clase B para 300 hosts se desperdician **65 234** de 65 534 direcciones: el 99,5 %. Por eso CIDR, en 1993.
5. **El `/n` y el número mágico** ( `RFC 1878` ) son el atajo del subnetting: el menor número de la tabla que sea ≥ lo que necesitás.
6. **CIDR** ( `RFC 4632` ) liberó la longitud del prefijo, volvió la máscara **local a cada enlace** y rehabilitó la **sumarización**, sin la cual la tabla de Internet no cabría.
7. Los rangos especiales existen **cada uno por un motivo semántico**: `0/8` = inexistencia, `127/8` = yo mismo, `169.254/16` = autoconfiguración en el enlace, `224/4` = grupo, `255.255.255.255` = broadcast local. Las **privadas** existen porque IPv4 se agotó y porque no todo tiene que ser alcanzable desde todo el mundo.
8. El **checksum** es la suma del complemento a uno de las palabras de 16 bits de la cabecera, **con acarreo**, y se verifica porque al verificar da `0xFFFF`. Cubre **sólo la cabecera**.

## ✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

**1.** ¿Por qué existe el campo IHL si la cabecera IPv4 "son 20 bytes"?

<details>
<summary>Respuesta</summary>
</details>

**2.** Calculá la red, el rango de hosts y el broadcast de `192.168.10.55` con máscara `/26`.

<details>
<summary>Respuesta</summary>
</details>

**3.** ¿Qué clase tiene `172.16.40.9` y qué máscara lleva "por defecto"?

<details>
<summary>Respuesta</summary>
</details>

**4.** ¿Por qué se agotaron las clases B? Poné los números.

<details>
<summary>Respuesta</summary>
</details>

**5.** ¿Por qué DHCP autoasigna `169.254.x.x` y no `0.0.0.0` ni `192.168.1.5`?

▶ Respuesta

**6.** Calculá el checksum de la cabecera de un datagrama con longitud total 60, TTL 64, protocolo TCP, origen `192.168.10.55` y destino `219.41.31.21`.

▶ Respuesta

**7.** Dos `/24` contiguos, `192.168.1.0/24` y `192.168.2.0/24`. ¿Se pueden agrupar en un solo prefijo? ¿Por qué?

▶ Respuesta

**8.** ¿Y si las dos subredes fueran `192.168.108.0/24` y `192.168.109.0/24`? ¿Se pueden agrupar?

▶ Respuesta

## 🔗 Referencias

- **Examen `eirc1003` / `sirc1003`** — Problema 1 (30 pts): tres subredes `/24` con R1 y R2, tabla de enrutamiento de routers y de hosts, y el renumerado mínimo al reemplazar R2 por un switch. El mejor ejemplo de **sumarización** del banco.
- **Examen `eirc0912` / `sirc0912`** — Problema 1: tres clases C y la agregación de `192.168.108.0/24` + `192.168.109.0/24` en una `/23`.
- **Examen `eirc1103` / `sirc1103`** — Problema 2d: dividir un `/24` en dos subredes de tamaños distintos. · **Examen `eirc0812` / `sirc0812`** — P2: plan de numeración.
- **Examen `eirc0712` / `sirc0712`** — Problema 2a: ARP entre dos hosts y un router. · **Examen `eirc0608`** — P4: `200.40.224.0/25` subdividido en redes de 18, 11 y 9 hosts.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red" — §5.4 el protocolo IP: la cabecera.
- **RFC 791** — Internet Protocol (1981): el formato de la cabecera, el TTL, la fragmentación y el mínimo de 576 bytes.
- **RFC 1122** — Requirements for Internet Hosts (1981): comportamiento de host, incluido el mínimo de 576 bytes.
- **RFC 1812** — Requirements for IP Version 4 Routers (1995): qué debe hacer un router con la cabecera, incluido el checksum y el TTL.
- **RFC 1878** — Requirements for IP Address Reservation (1996): **el anexo con el "número mágico"**, la tabla de hosts por prefijo.
- **RFC 4632** — CIDR: An Address Assignment Plan for IPv4 (2006): el fin de las clases, el prefijo libre y la sumarización.
- **RFC 8200** — IPv6 (2017): la cabecera fija de 40 bytes, sin IHL ni opciones, y el campo *Flow Label*.
- ⚠️ **RFC que hay que verificar antes de citar en un examen** (no figuran en `ACTUALIZACIONES.md`): `RFC 1918` (direcciones privadas), `RFC 950` (subnetting, 1984, obsoleta), `RFC 3927` (link-local/APIPA),

`RFC 6598` (CGNAT, `100.64/10`) y `RFC 7126` (cabecera IPv4 con IHL). Confirmalos en `rfc-editor.org`.

- **Clase siguiente:** 27 — Subnetting, método paso a paso